

Weekly Progress Report

Name: *Blake Simpson* **Week:** *2 (Setup & Background)* **Track:** *Track 3 — Agent*

WHAT I DID THIS WEEK

I worked with Claude to set up the project environment, worked on the LLM integration which I chose to use claude CLI. Im taking inspiration from how openclaw makes use of the claude subscription login to do agentic tasks. This allows me to use my claude subscription and not pay for the api usage. I also worked with claude on getting my deps installed, LangGraph for agent flows, requests, pydantic, pypdf, python-dotenv.

WHAT WORKED

- The Remote API is public and doesnt require and auth. Tested it and it works and pulls live postings
- Reusing my claude CLI login also worked and I verified it with the `verify_setup.py` script.
- All of the deps install in the py env

WHAT FAILED OR SURPRISED ME

- When working with Claude it found the below issues with using my claude CLI:
 - The `claude` CLI prints rate-limit / out-of-quota messages as **plain text and still exits 0** — so a naive "check the exit code" would treat a throttled run as success. I had to scan stdout/stderr for markers (`"rate limit"` , `"out of extra usage"`) *before* trusting the return code.
 - Claude's output isn't clean JSON — it wraps the object in prose, so I had to pull the first `{...}` out with a regex (`extract_json`) instead of calling `json.loads` directly.
 - Running under cron/systemd, a bare `claude` command fails ("command not found") because those run with a minimal PATH — I learned I have to configure the **absolute** binary path via `CLAUDE_PATH` .

- My interpretation of this is that claude's output is wrapped in text that is not json and so the returned json object needs to be cut out first then can be processed with a python json function.

WHAT I LEARNED

I learned how to use the claude cli auth in my app as well as how to cleanly extract json objects from its responses.

EVIDENCE OF PROGRESS

- GitHub commits: `ba503d9` (initial scaffold + README) and `e7bde8d` (switch to claude CLI subprocess auth): can be found here: <https://github.com/EXC3ll3NTrhyTHM/agent-workflow>
- python scripts/verify_setup.py output:

```
blakesimpson@Blakes-MacBook-Air agent-workflow % python scripts/verify_setup.py
Job Scout Agent – setup verification
```

```
[1/2] Remote API (job data) ...
      OK – reachable. Sample posting: 'Mid/Senior AI Cinematic Video Editor'
[2/2] Claude CLI (subprocess auth) ...
      binary: /Users/blakesimpson/.local/bin/claude
      OK – Claude replied: 'pong'
```

All checks passed (or skipped). You're ready for Week 3.

PLAN FOR NEXT WEEK

- Wire everything up so that it can make a call using a submitted resume.
 - load a submitted resume, query remote, and have claude score each posting against the resume so that a ranked list is produced.
- Create a baseline that future updates to the app can be measured against; 10 to 15 postings with 0 to 10 score so I can see how the scores drift with updates.
- In order to ensure this I will create a multi-criteria rubric to grade the job scout results which will produce a final overall grading.
- Revisit scope and determine whether the "9 and above send an email alert" threshold should remain or should be updated and then determine if the UI is still optional or if I have time to

commit to it

OPTIONAL: BLOCKERS / QUESTIONS FOR ZOOM